
Rapport Technique

Pipeline Kafka - Analytics temps reel - Dashboard live

AUTEURS	Adam Beloucif, Emilien Morice
FORMATION	M1 Data Engineering & IA - EFREI Paris
MODULE	Real-Time Engineering - S9 2025-2026
DATE	Juin 2026
CONTEXTE	Projet realise a 2 personnes (consigne prevue pour 4)

INTRODUCTION

Ce rapport presente l'architecture, les choix techniques et les resultats du projet Crypto Market Monitor, realise dans le cadre du module Real-Time Engineering du M1 Data Engineering et IA de l'EFREI Paris (2025-2026).

L'objectif : construire un pipeline de streaming bout-en-bout qui ingere des transactions crypto en direct depuis Binance et Coinbase, les traite via Apache Kafka, calcule des metriques analytiques en fenetre glissante (VWAP, SMA-20, z-score) et pousse les resultats vers un dashboard live via Socket.IO.

100M+ transactions/jour (Binance)	< 1s latence cible detection anomalie	3 sources Binance + Coinbase	4 services independants
--	---	---	--

ARCHITECTURE

Le systeme suit un pipeline lineaire : collecte WebSocket -> Kafka -> traitement analytique -> API REST/WebSocket -> dashboard. Chaque service est isole dans un conteneur Docker independant.

Service	Role	Technologies
Ingester	Clients WS Binance + Coinbase, normalisation, production Kafka	Node.js, TypeScript, ws, kafkajs
Kafka	Tampon decoupling, tolerance pannes, multi-consommateurs	Apache Kafka 3.7 KRaft
Processor	Consomme trades, calcule VWAP SMA-20 z-score variance	Node.js, TypeScript, kafkajs
API	Agregation, endpoints REST, push WebSocket live	Express, Socket.IO, helmet
Dashboard	Graphiques Chart.js, flux anomalies, i18n FR/EN	HTML, CSS, JS, Chart.js 4

```
WS Sources -> Ingester -> Kafka [crypto-trades] -> Processor -> Kafka [crypto-metrics]
-> API -> Socket.IO -> Dashboard
```

Le partitionnement par symbole (cle = BTC-USDT, ETH-USDT, BTC-USD) garantit l'ordre des messages par paire. La retention est de 1 heure. En cas de crash du processor, il relira le topic depuis son dernier offset consomme.

APACHE KAFKA

Kafka 3.7 en mode KRaft elimine Zookeeper. Les metadonnees du cluster sont geres via Raft interne, ce qui simplifie le deploiement (1 conteneur + 1 volume) et reduit la surface d'attaque operationnelle.

Topic	Description	Partitions	Retention
crypto-trades	Transactions brutes normalisees (interface Trade)	3	1h
crypto-metrics	Metriques calculees par symbole (VWAP, SMA, etc.)	3	1h

SERVICES NODE.JS

L'ingester maintient des connexions WebSocket persistantes et normalise chaque message vers une interface Trade commune avant de le produire sur Kafka.

Metrique	Formule	Fenetre	Objectif
VWAP	$\text{sum}(\text{prix} * \text{qte}) / \text{sum}(\text{qte})$	1 min	Prix moyen pondere par volume
SMA-20	moyenne des 20 derniers prix	20 trades	Lissage de la tendance
Z-Score	$(\text{valeur} - \mu) / \sigma$	1 min	Anomalie si $> 2.5 \sigma$
Var. 1min	$(\text{prix} - \text{prix0}) / \text{prix0}$	1 min	Performance instantanee
Var. 5min	$(\text{prix} - \text{prix0}) / \text{prix0}$	5 min	Performance moyen terme
High/Low	max / min des prix	1 min	Amplitude de la bougie

Buffer circulaire : max 1000 trades / symbole, eviction TTL 10 minutes

L'API expose 5 endpoints REST et 5 types d'evenements Socket.IO. Elle maintient l'etat courant en memoire : dernieres metriques + historique 200 points + 50 anomalies.

Endpoint / Evenement	Type	Description
GET /api/health	REST	Sante du service
GET /api/metrics	REST	Toutes les metriques
GET /api/metrics/:symbol	REST	Metriques par paire
GET /api/history/:symbol	REST	Historique 200 points
GET /api/anomalies	REST	50 dernieres anomalies
metrics:update	Socket.IO	Push nouvelles metriques
anomaly:detected	Socket.IO	Push alerte z-score
server:stats	Socket.IO	Stats connexions / msg/s

DASHBOARD

Application vanilla HTML/CSS/JS connectee via Socket.IO. Aucun framework front-end pour minimiser les dependances et maximiser la performance de chargement.

Palette brand EFREI officielle : fond navy #163767, accent rose #ff43b8, bleu #3653a0. Barre gradient EFREI en haut de page. Logos SVG officiels Bitcoin, Ethereum, Coinbase. Typo Inter (Google Fonts).

Si le backend est inaccessible apres 3 secondes, le mode demo s'active. Donnees simulees realistes (BTC ~67k, ETH ~3.5k), mise a jour 800ms. Badge DEMO rose dans le header.

INTERNATIONALISATION

Module vanilla JS IIFE (I18n) sans dependance externe. Charge les JSON depuis i18n/{locale}.json, persiste le choix en localStorage, applique les traductions via attributs data-i18n.

SECURITE

Composant	Protection
helmet	HTTP headers : CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
express-rate-limit	100 req / 15min / IP - protection brute-force et DDoS applicatif
zod	Validation stricte des variables d'environnement - arret si manquante/invalid
CORS strict	Whitelist explicite des origines - pas de wildcard (*)
morgan	Logging HTTP avec rotation - aucune stack trace exposee cote client
Docker non-root	Tous les conteneurs tournent en utilisateur appuser (UID 1000)
.dockerignore	Exclusion des .env, node_modules, secrets des images Docker
Secrets env vars	Zero secret en dur - .env gitignore, patterns dans .gitignore

DEPLOIEMENT

Option	Commande	Usage
Lanceur .exe	node launcher/src/index.mjs (compile : npm run build:win)	Demo, machines sans Docker Compose
Image tout-en-un	docker run -p 8080:8080 crypto-monitor	Demo standalone, 1 seul conteneur
Docker Compose	docker-compose up --build -d	Developpement, debug, Kafka UI :8090

Variable	Defaut	Description
KAFKA_BROKERS	localhost:9092	Bootstrap servers Kafka
KAFKA_TOPIC_TRADES	crypto-trades	Topic transactions brutes
KAFKA_TOPIC_METRICS	crypto-metrics	Topic metriques calculees
API_PORT	3001	Port du serveur API
SMA_WINDOW	20	Fenetre SMA (nombre de trades)
ANOMALY_ZSCORE_THRESHOLD	0.5	Seuil z-score alerte
HISTORY_WINDOW_SECONDS	300	Retention historique (5 min)

BILAN

Crypto Market Monitor - Rapport Technique

Adam Beloucif, Emilien Morice - M1 Data Engineering & IA - EFREI Paris - 2025-2026